

Módulo: Programación Multimedia y dispositivos móviles.
Unidad de Trabajo N°1: Introducción al desarrollo de aplicaciones móviles.
Actividad de Clase 5: Base de Datos SQLite.

1. Objetivo general.

Conocer, manejar y generar aplicaciones que trabaje con la base de datos SQLite.

2. Metodología.

El alumnado realizará la app de forma individual.

3. Descripción.

Base de Datos SQLite.

SQLite es un motor de bases de datos muy popular en la actualidad porque ofrece características tan interesantes como su pequeño tamaño, no necesitar servidor, precisar poca configuración, ser transaccional y por supuesto ser de código libre.

En Android, la forma típica para crear, actualizar, y conectar con una base de datos SQLite será a través de una clase auxiliar llamada **SQLiteOpenHelper**, mejor dicho, de una clase propia que derive de ella y que debemos personalizar para adaptarnos a las necesidades concretas de nuestra aplicación.

La clase **SQLiteOpenHelper** tiene tan sólo un constructor, que normalmente no necesitaremos sobrescribir, y dos métodos abstractos, **onCreate()** y **onUpgrade()**, que deberemos personalizar con el código necesario para crear nuestra base de datos y para actualizar su estructura respectivamente.

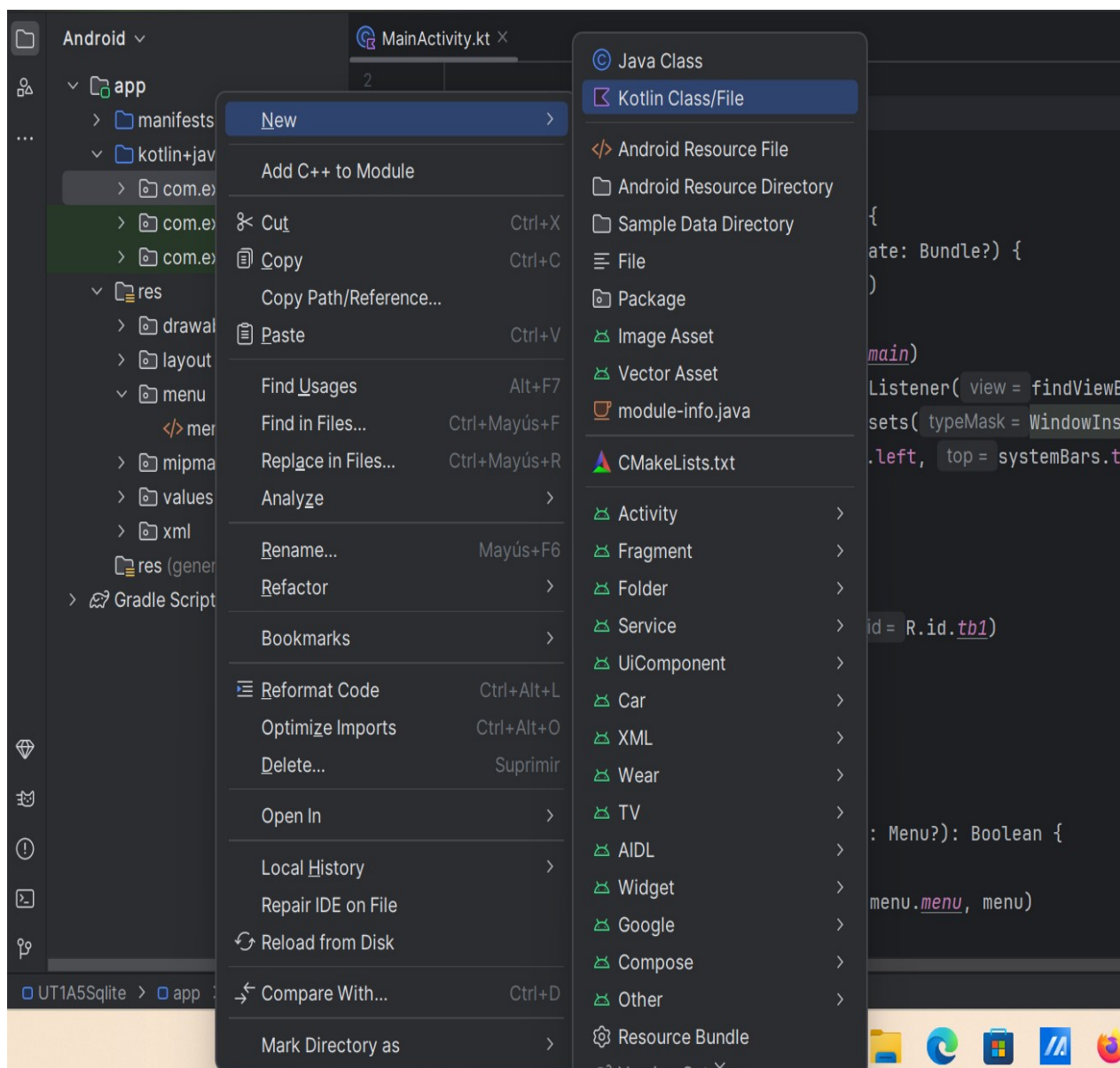
Creación de un aplicación que usando una Base de Datos SQLite.

Realiza lo siguiente:

1) Para ver el uso de **SQLite**, vamos a crear una aplicación de ejemplo. Para ello crea un nuevo Proyecto. En este ejemplo de proyecto trabajaremos con una base de datos que almacenará datos de Equipos, pero tu puedes adaptarla al ejemplo que quieras. Con una sola tabla Equipos que contendrá los siguientes campos: Código, Nombre y Presupuesto. Si usas otro ejemplo utiliza la tabla y los campos adecuados.

2) Para ellos, vamos a crear una clase que herede de **SQLiteOpenHelper** que llamaremos **EquiposSQLiteOpenHelper**, donde sobrescribiremos los métodos **onCreate()** y **onUpgrade()** para adaptarlos a la estructura de datos indicada:

3) Creamos la clase:



4) Una vez creada la clase hacemos que herede de **SQLiteOpenHelper**. En el constructor definimos el contexto, nombre de la base de datos, CursorFactory que se usa para la devolución de la clase Cursor para realizar consultas y la versión de la base de datos. También, definimos un string que contiene la sentencia SQL donde creamos la tabla. Y luego sobrescribimos los métodos onCreate() y onUpgrade(). En onCreate() ejecutamos la setencias SQL donde creamos la tabla. En onUpgrade() en este momento no implementes nada.

```
EquiposSQLiteOpenHelper.kt
package com.example.ut1_a5_sqlite

import android.content.Context
import android.database.sqlite.SQLiteDatabase
import android.database.sqlite.SQLiteOpenHelper

class EquiposSQLiteOpenHelper(context: Context, nombre: String, factory: SQLiteDatabase.CursorFactory?, version: Int): SQLiteOpenHelper(context = context, name = nombre, factory, version) {

    override fun onCreate(p0: SQLiteDatabase?) {
        p0!!.execSQL("CREATE TABLE Equipos(codigo int primary key, nombre text, presupuesto number)")
    }

    override fun onUpgrade(p0: SQLiteDatabase?, p1: Int, p2: Int) {
        TODO("reason = \"Not yet implemented\"")
    }
}
```

- 5) Si queremos podemos añadir a nuestra clase las funciones para insertar, buscar, actualizar y eliminar (borrar) por código.
- 6) Para ello realizaremos una función para cada una de las operaciones que vamos a realizar.

Insertar

- 7) Insertar un equipo.

```
fun insertar(codigo: Int, nombre: String, presupuesto: Double): Int {
    val db: SQLiteDatabase = this.writableDatabase
    val registro = ContentValues()
    registro.put("codigo", codigo)
    registro.put("nombre", nombre)
    registro.put("presupuesto", presupuesto)
    val insertdo = db.insert(table = "Equipos", nullColumnHack = null, values = registro)
    return insertdo.toInt()
}
```

Primer creamos un objeto de la clase **SQLiteDatabase** accediendo a la propiedad **writableDatabase** (la base de datos se abre en modo lectura y escritura). A continuación, creamos un objeto de la clase **ContentValues** y mediante el método **put** inicializamos todos los campos a cargar.

Seguidamente llamamos al método **insert** de la clase **SQLiteDatabase** pasando en el **primer parámetro** el nombre de la tabla, como **segundo parámetro** un null (si no pusieramos null era para permitir insertar filas vacías) y por último el objeto de la clase **ContentValues** ya inicializado (este método es el que provoca que se inserte una nueva fila en la tabla Equipos en la base de datos llamada Equipos), registrando el resultado en una variable que es el resultado que devolverá nuestra función.

Buscar por código

8) Ahora vamos a buscar por código que al ser nuestra clave primaria nos debe devolver un sólo registro o ninguno.

```
fun buscarporCodigo(codigo: Int): Cursor {  
    val db: SQLiteDatabase = this.writableDatabase  
    val registros = db.rawQuery("SELECT nombre, presupuesto FROM Equipos WHERE codigo=$codigo", selectionArgs = null)  
    return registros  
}
```

Creamos un objeto de la clase **SQLiteDatabase**. Seguidamente definimos una variable de la clase **Cursor** llamada registros y la inicializamos con el valor devuelto por el método llamado **rawQuery**. Y devolveremos la clase **Cursor** que en este caso almacena una fila o cero filas (una en caso que hayamos ingresado un código existente en la tabla equipos).

Modificar un registro por código

9) Ahora vamos a ver el código que modifica un registro (todos los campos, excepto el código) por código.

```
fun modificaRegistro(codigo: Int, nombre: String, presupuesto: Double): Int {  
    val db: SQLiteDatabase = this.writableDatabase  
    val register = ContentValues()  
    register.put("nombre", nombre)  
    register.put("presupuesto", presupuesto)  
    val nactualiza = db.update("Equipos", register, "codigo=$codigo", whereArgs = null)  
    return nactualiza  
}
```

10) Como en casos anteriores, creamos un objeto de la clase **SQLiteDatabase**. Luego creamos un objeto de la clase **ContentValues** y mediante el método **put** almacenamos los valores para cada campo que será modificado. Luego se llama al método **update** de la clase **SQLiteDatabase** pasando el nombre de la tabla, el objeto de la clase **ContentValues** y la condición del **where** (el cuarto parámetro en este ejemplo no se lo emplea). Y devolvemos el número de registros actualizados, al hacerlo por el código que es la clave primaria debería ser 1 ó 0.

Borrar un registro por código

11) A continuación, puedes ver el código que borra un registro por código:

```
fun borraRegistro(codigo: Int): Int {  
    val db: SQLiteDatabase = this.writableDatabase  
    val neliminados = db.delete( table = "Equipos", whereClause = "codigo=$codigo", whereArgs = null)  
    return neliminados  
}
```

Donde ejecutamos el delete, seleccionando la tabla y borramos por el código. Y devolvemos el número de registros borrados que al hacerlo por el código (que es la clave primaria) será 1 o 0.

12) Ahora implementemos la interfaz visual para resolver nuestro problema.

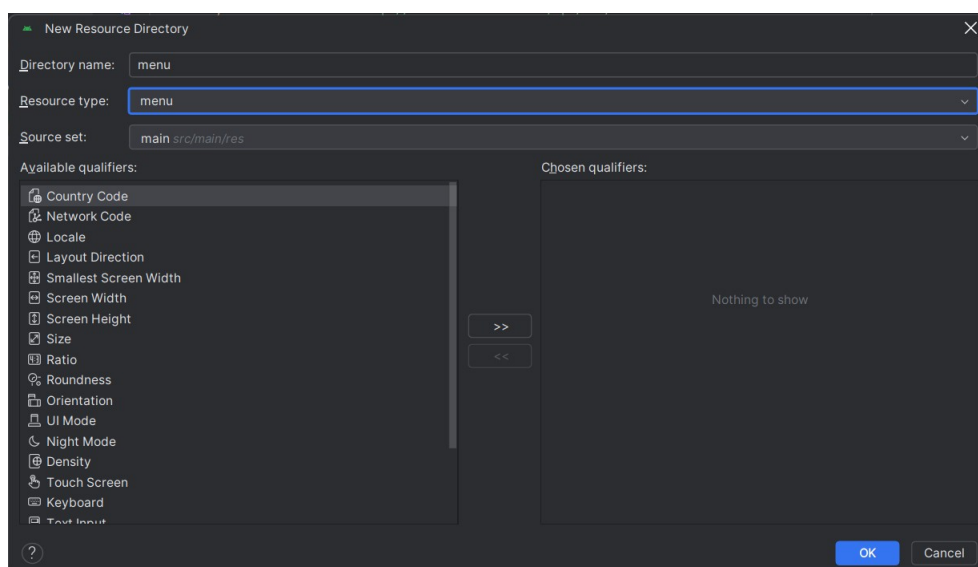
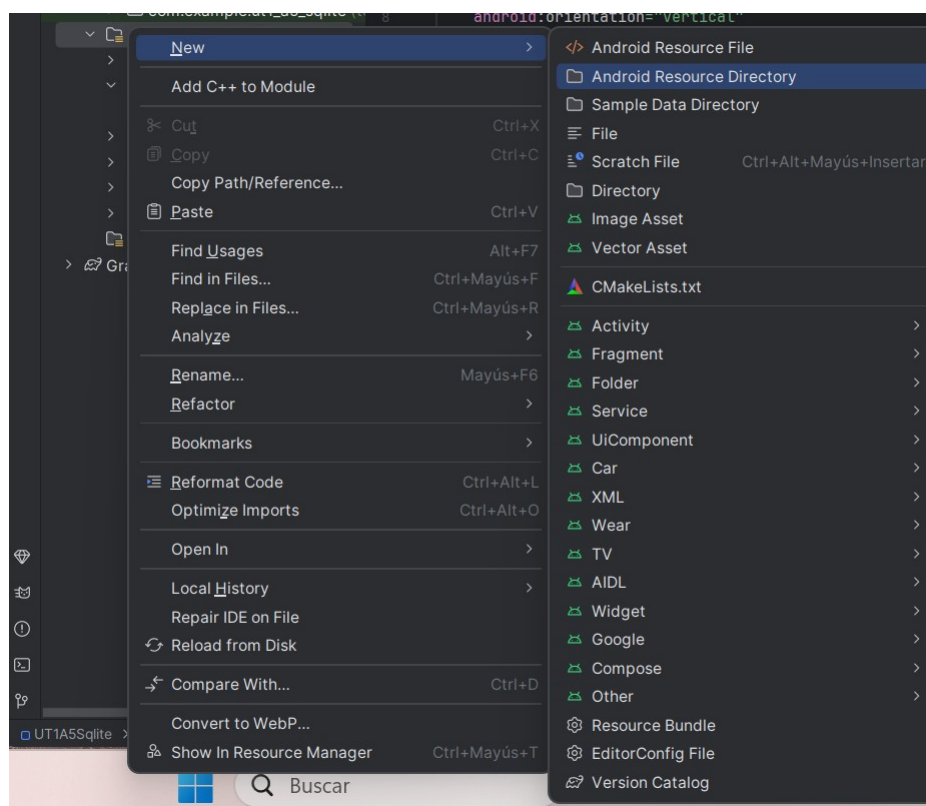
Menu

Te propongo que las opciones de la base de datos las hagas con un Menú. Si quieres hacerla con botones como hemos visto en actividades anteriores y no realizarías esta parte y pasarías a más adelante.

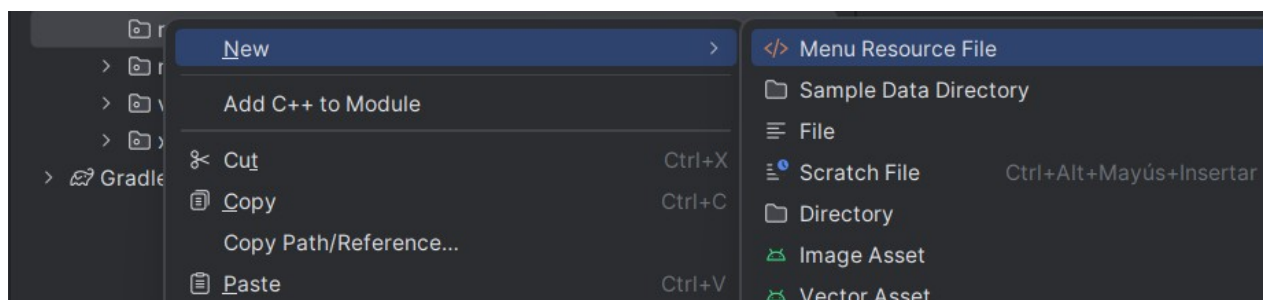
13) En las últimas versiones no tenemos activada la **Toolbar**, vamos a añadirla. En el fichero **activity_main.xml** añadiremos la **Toolbar**. Para ello añade lo siguiente al fichero xml en cuestión. También si quieres puedes añadirlo en diseñador gráfico.

```
<androidx.appcompat.widget.Toolbar  
    android:id="@+id/tb1"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:background="?attr/colorPrimary"  
    android:minHeight="?attr/actionBarSize"  
    android:theme="?attr/actionBarTheme" />
```

14) Ahora vamos a crear un menú, en la carpeta **res** como un nuevo recurso, de tipo menú. Para ello con el botón secundario (derecho generalmente) sobre la carpeta **res** y pulsamos sobre **New** → **Android Resource Directory**



15) Luego creamos el XML, para ello dentro del directorio menu, creamos un Menu Resource File



16) Una vez creado podemos modificarlo de forma manual o usar el diseñador como hacíamos con las layouts. Añade cinco items al menú uno para Insertar, Buscar, Actualizar, Borrar y Todos, si quieres puedes añadir más opciones. Coloca el id y el título (lo que aparecerá en el menú) que quieras. A continuación puedes ver una imagen del fichero XML con las opciones del menú con sus id y su título correspondiente.

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <menu xmlns:android="http://schemas.android.com/apk/res/android">
3
4     <item
5         android:id="@+id/insertar"
6         android:title="Insertar" />
7     <item
8         android:id="@+id/buscar"
9         android:title="Buscar por código" />
10    <item
11        android:id="@+id/actualizar"
12        android:title="Actualizar por código" />
13    <item
14        android:id="@+id/borrar"
15        android:title="Borrar por código" />
16    <item
17        android:id="@+id/todos"
18        android:title="Mostrar Todos" />
19
20 </menu>

```

17) A continuación tenemos que vincular el menú y manejar las diferentes opciones. Para ello debemos escribir el código en Kotlin que debe ser similar al siguiente. En primer lugar vamos a activar la **ToolBar** para que se muestre en nuestra aplicación. En la función **onCreate** de nuestra **MainActivity** ponemos el siguiente código:

```

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    enableEdgeToEdge()
    setContentView(R.layout.activity_main)
    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v, insets ->
        val systemBars = insets.getInsets(typeMask = WindowInsetsCompat.Type.systemBars())
        v.setPadding(left = systemBars.left, top = systemBars.top, right = systemBars.right, bottom = systemBars.bottom)
    }

    // Accedemos a la Toolbar
    val tbi = findViewById<Toolbar>(R.id.tbi)
    // Activamos la Toolbar
    setSupportActionBar(tbi)
}

```

18) Vamos con la implementación del menú. En primer lugar creamos las opciones, para ello dentro de la clase **MainActivity** (pero fuera del método onCreate) sobrescribe (override) el método **onCreateOptionsMenu** donde mostramos el menú (método inflate que muestra el contenido de un fichero xml como es el del menú). Lo puedes hacer con el botón derecho del ratón y la opción **Generate... Override Methods**

```

// Creamos el menú y sus opciones
override fun onCreateOptionsMenu(menu: Menu?): Boolean {
    super.onCreateOptionsMenu(menu)
    menuInflater.inflate(R.menu.menu, menu)
    return true
}

```

19) El código kotlin para la selección del ítem puede ser similar al que aparece a continuación. Dicho código también estará dentro de nuestra clase MainActivity. (pero fuera del método onCreate) sobrescribe (override) el método **onOptionsItemSelected**. Donde hemos creado una función para cada una de las opciones de las operaciones que realizaremos con la Base de Datos. Que se ejecutarán cuando seleccionemos la opción del menú.

```

// Manejamos la selección sobre los ítems del menú
override fun onOptionsItemSelected(item: MenuItem): Boolean {
    when(item.itemId) {
        R.id.insertar -> {
            insertar()
            limpiar()
        }

        R.id.buscar -> {
            buscarporcodigo()
        }

        R.id.actualizar -> {
            actualizar()
        }

        R.id.borrar -> {
            borrar()
            limpiar()
        }

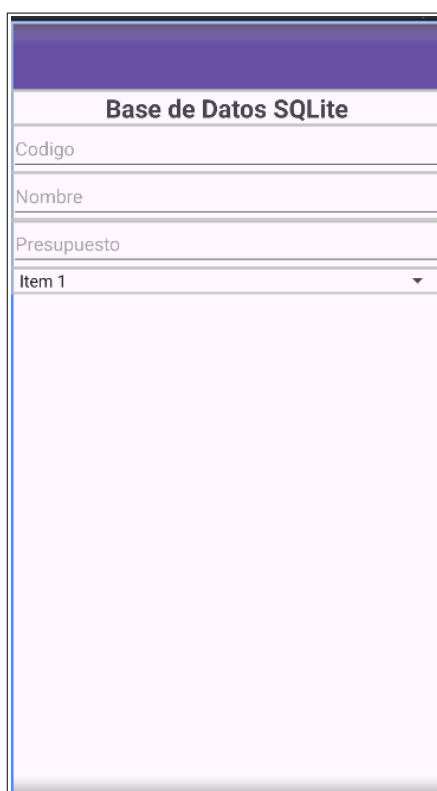
        R.id.todos -> {
            todos()
        }
    }

    return super.onOptionsItemSelected(item)
}

```

MainActivity

20) Debemos crear en nuestro archivo **activity_main.xml** . Donde debes tener un TextView (que será el título de aplicación y no modificaremos, tres EditText (uno para cada uno de los campos de la tabla, si tiene más puedes añadir más). Y un Spinner. Caso de hacerlo con botones en vez del menú aquí los añadirías, si lo haces con Menú no tienes porque añadirlo. A continuación, puedes observar una imagen con una propuesta de interfaz (esta es una propuesta tienes la libertad de diseñarlo a tu gusto siempre que respetes los campos de la base de datos y los botones en el caso de usarlos, si usas Menú no hacen falta botones). Esta diseñado con un LinearLayout para todos los componentes y varios para los botones.



21) Ahora vamos a **MainActivity.kt** para codificar las diferentes acciones. En primer lugar accedemos a cada uno de componentes que hemos insertado en nuestro fichero **activity_main.xml** . Ya sabes como hacerlo para ello puedes utilizar **findViewById** o **ViewBinding**. Al utilizar diferentes funciones debes declararlos a nivel de clase o pasarlo como parámetros.

22) A continuación, comenzamos con cada una de las funciones que realizan las operaciones sobre la base de datos. En caso de usar botones y no menú debes crear los listener para cada botón. Comenzamos con **insertar**.

Insertar

```
private fun insertar() {  
    val bd = EquiposSQLiteOpenHelper( contexto = this, nombre = "Equipos", factory = null, version = 1)  
    val resultado = bd.insertar( codigo = etcodigo.text.toString().toInt(), nombre = etnombre.text.toString(), presupuesto = etpresu.text.toString().toDouble())  
    if (resultado!=0) {  
        muestraMensaje( mensaje = "Registro insertado")  
    } else {  
        muestraMensaje( mensaje = "Registro NO insertado")  
    }  
}
```

23) Creamos un objeto de la clase **EquiposSQLiteOpenHelper** y le pasamos al constructor **this** (referencia del Activity actual), **"Equipos"** (es el nombre de la base de datos que crearemos en el caso que no exista) luego pasamos null (indicando que vamos a usar la versión de cursor por defecto) y un uno indicando que es la primer versión de la base de datos (en caso que cambiemos la estructura o agreguemos tablas por ejemplo podemos pasar un dos en lugar de un uno para que se ejecute el método **onUpgrade** donde indicamos la nueva estructura de la base de datos). A continuación definimos una constante (definido con val) resultado y ejecutamos el método insertar que creamos en la clase que donde definimos la base de datos. Según nos devuelva el método sabremos si se ha insertado o no el registro en la Base de datos. En caso afirmativo muestra el mensaje en el Toast de Registro insertado y en caso contrario Registro no insertado.

Nota: Ten en cuenta que en este ejemplo no he puesto control de errores sobre los campos que tendrías que hacerlos.

La función **muestraMensaje** es una función donde tiene como parámetro de entrada el mensaje a mostrar en un Toast. De esa forma. La función no está realizada la tendrían que realizar cada uno de ustedes.

Buscar por código

24) Ahora vamos a ver el código que busca un registro por código (debe existir un sólo registro).

```
private fun buscarporcodigo() {  
    val db = EquiposSQLiteOpenHelper( contexto = this, nombre = "Equipos", factory = null, version = 1)  
    val regis = db.buscarporCodigo( codigo = etcodigo.text.toString().toInt())  
    if (regis.moveToFirst()) {  
        etnombre.setText(regis.getString( p0 = 0))  
        etpresu.setText(regis.getDouble( p0 = 1).toString())  
    } else {  
        muestraMensaje( mensaje = "Equipo NO encontrado")  
        limpiar()  
    }  
}
```

- 25) Como en insertar lo primero que hacemos es crear un objeto de la clase **EquiposSQLiteOpenHelper** luego creamos un objeto regis que recibirá el resultado de ejecutar el método **buscarporCodigo**. Luego llamamos al método **moveToFirst()** de la clase **Cursor** y retorna true en caso de existir un equipo con el código ingresado, en caso contrario retorna cero.

Para recuperar los datos propiamente dichos que queremos consultar llamamos al método **getString** o **getDouble** y le pasamos la posición del campo a recuperar (comienza a numerarse en cero, en este ejemplo la columna cero representa el campo **nombre** y la columna 1 representa el campo **presupuesto**) que mostramos en los **EditText** correspondientes utilizando el método **setText**, pasando los valores a **String** con el método **toString()**. En caso de existir ningún registro con dicho código pues mostramos el mensaje y ejecutamos un método que he creado que se llama limpiar que limpia los **EditText**. Tú deberías hacer uno.

Modificar un registro por código

- 26) Ahora vamos a ver el código que modifica un registro por código.

```
private fun actualizar() {  
    val db = EquiposSQLiteOpenHelper( contexto = this, nombre = "Equipos", factory = null, version = 1)  
    val actualizados = db.modificaRegistro( codigo = etcodigo.text.toString().toInt(), nombre = etnombre.text.toString(), presupuesto = etpresu.text.toString().toDouble())  
    if (actualizados==1) {  
        muestraMensaje( mensaje = "Equipo actualizado")  
    } else {  
        muestraMensaje( mensaje = "Equipo NO actualizado")  
    }  
}
```

- 27) Como en las opciones anteriores creamos un objeto de la clase **EquiposSQLiteOpenHelper** y le pasamos al constructor **this** (referencia del Activity actual), **"Equipos"** (es el nombre de la base de datos que crearemos en el caso que no exista) luego pasamos null (indicando que vamos a usar la versión de cursor por defecto) y un uno indicando que es la primer versión de la base de datos.

28) Luego definimos una constante **actualizados** y ejecutamos el método **modificaRegistro** que creamos en la clase que donde definimos la base de datos. Según nos devuelva el método sabremos si se ha modificado o no el registro en la Base de datos. En caso afirmativo muestra el mensaje **Equipo actualizado** y en caso contrario **Equipo no actualizado**. Recuerda que deberías implementar el control de errores que no está implementado por simplificar la explicación.

Borrar un registro por código

- 29) A continuación, puedes ver el código que borrar un registro por código:

```
private fun borrar() {  
    val db = EquiposSQLiteOpenHelper( contexto = this, nombre = "Equipos", factory = null, version = 1)  
    val borrados = db.borraRegistro( codigo = etcodigo.text.toString().toInt())  
    if (borrados==1) {  
        muestraMensaje( mensaje = "Equipo eliminado")  
    } else {  
        muestraMensaje( mensaje = "Equipo NO eliminado")  
    }  
}
```

- 30) De nuevo creamos un objeto de la clase **EquiposSQLiteOpenHelper** y le pasamos al constructor **this** (referencia del Activity actual), **"Equipos"** (es el nombre de la base de datos que crearemos en el caso que no exista) luego pasamos null (indicando que vamos a usar la versión de cursor por defecto) y un uno indicando que es la primer versión de la base de datos.
- 31). Luego definimos una constante borrados y ejecutamos el método borraRegistro que creamos en la clase que donde definimos la base de datos. Según nos devuelva el método sabremos si se ha borrado o no el registro en la Base de datos. En caso afirmativo muestra el mensaje Equipo eliminado y en caso contrario Equipo no eliminado. Recuerda que deberías implementar el control de errores que no está implementado por simplificar la explicación
- 32) Ejecuta la aplicación y comprueba su funcionamiento. Puedes ver un ejemplo de funcionamiento en el siguiente video:

<https://drive.google.com/file/d/1b-Ok-6MUChqmHVX4rgSyfpG7tMb5TVGa/view?usp=sharing>

Mostrar todos.

- 33) Con esto ya estaría terminada la actividad ya habríamos hecho la típica aplicación con Base de Datos que hace **CRUD** "Crear, Leer, Actualizar y Borrar" (del original en inglés: Create, Read, Update and Delete)
- 34) Si quieres puedes ver como cargar datos en un Spinner desde una tabla de SQLite. Para ello tenemos la opción (o botón) **Mostrar Todos** y un **Spinner**. Al hacer click sobre la opción (o botón) se ejecuta un consulta que obtiene todos los datos y almacenamos los nombres de los Equipos que guardamos en un ArrayList que será luego de donde cargará los datos el Adaptador para mostrarlos en el Spinner.
- 35) A continuación puede ver el código correspondiente, a la opción Mostrar todos (o Listener si elegiste hacerlo utilizando un botón) y del Spinner:

```
private fun todos() {
    val db = EquiposSQLiteOpenHelper( contexto = this, nombre = "Equipos", factory = null, version = 1)
    val listanames = ArrayList<String>()
    val toditos = db.mostrarTodos()
    while (toditos.moveToNext()) {
        listanames.add(toditos.getString( p0 = 0))
    }
    val adaptador = ArrayAdapter( context = this, resource = androidx.appcompat.R.layout.support_simple_spinner_dropdown_item, objects = listanames)
    epinall.adapter = adaptador
}
```

36) Al igual que las opciones anteriores, creamos un objeto de la clase **EquiposSQLiteOpenHelper** y le pasamos al constructor **this** (referencia del Activity actual), **"Equipos"** (es el nombre de la base de datos que crearemos en el caso que no exista) luego pasamos null (indicando que vamos a usar la versión de cursor por defecto) y un uno indicando que es la primer versión de la base de datos.

37) Definimos el ArrayList de String, creamos un objeto donde la función que creamos en nuestra clase Base de Datos nos devuelve todos los nombres de los equipos que están insertados en la Base de datos y luego la recorremos y añadimos cada nombre al ArrayList.

38) Por último creamos el adaptador y lo vinculamos al adaptador del **spinner**.

39) Ahora si queremos podemos manejar el evento de cambio del spinner, para ello en el método onCreate manejas el Listener sobre el spinner, puedes verlo en la siguiente imagen, aunque ya lo vimos en la actividad anterior. En este caso mostramos el seleccionado en un Toast utilizando la función muestraMensaje.

```
epinall = findViewById( id = R.id.espinall)

epinall.onItemSelectedListener = object : AdapterView.OnItemSelectedListener {
    override fun onItemSelected(
        p0: AdapterView<*>?,
        p1: View?,
        p2: Int,
        p3: Long
    ) {
        muestraMensaje( mensaje = "Has seleccionado ${p0!!.getItemAtPosition( position = p2)}")
    }

    override fun onNothingSelected(p0: AdapterView<*>?) {
        TODO( reason = "Not yet implemented")
    }
}
```

40) Puedes observar la App funcionando con la modificación realizada (Mostrar todos en el Spinner.)

<https://drive.google.com/file/d/1VfSyK5zFiLe0-GYXH5iGXVxTDQla0EKi/view?usp=sharing>

Opciones para ejecutar sentencias SQL

41) Para insertar, modificar y borrar registros hemos utilizado los métodos **insert()**, **update()** y **delete()** proporcionados también con la clase **SQLiteDatabase**. Estos métodos permiten realizar las tareas de inserción, actualización y eliminación de registros de una forma algo más paramétrica, separando tablas, valores y condiciones en parámetros independientes de estos métodos.

42) Pero también podemos utilizar el método **execSQL()** de la clase **SQLiteDatabase**. Este método permite ejecutar cualquier sentencia SQL sobre la base de datos, siempre que ésta no devuelva resultados. Para ello, simplemente aportaremos como parámetro de entrada de este método la cadena de texto correspondiente con la sentencia SQL. Podemos ver los siguientes ejemplos:

//Insertar un registro

```
db.execSQL("INSERT INTO Usuarios (codigo,nombre) VALUES (9,'usuariopru') ");
```

//Eliminar un registro

```
db.execSQL("DELETE FROM Usuarios WHERE codigo=9 ");
```

//Actualizar un registro

```
db.execSQL("UPDATE Usuarios SET nombre='usunuevo' WHERE codigo=9 ");
```

43) Para el caso de las consultas vamos a tener dos opciones principales para recuperar registros de una base de datos SQLite en Android. La primera de ellas utilizando directamente un comando de selección SQL, y como segunda opción utilizando un método específico donde parametrizaremos la consulta a la base de datos.

44) En el primer caso, hemos usado el método **rawQuery()** de la clase **SQLiteDatabase**. Este método recibe directamente como parámetro un comando SQL completo, donde indicamos los campos a recuperar y los criterios de selección. El resultado de la consulta lo obtendremos en forma de cursor, que posteriormente podremos recorrer para procesar los registros recuperados

45) Como segunda opción para recuperar datos podemos utilizar el método **query()** de la clase **SQLiteDatabase**. Este método recibe varios parámetros: el nombre de la tabla, un array con los nombre de campos a recuperar, la cláusula WHERE, un array con los argumentos variables incluidos en el WHERE (si los hay, null en caso contrario), la cláusula GROUP BY si existe, la cláusula HAVING si existe, y por último la cláusula ORDER BY si existe. Opcionalmente, se puede incluir un parámetro al final más indicando el número máximo de registros que queremos que nos devuelva la consulta.